Response Letter

Manuscript ID: 26-3482

ADAPT: Adaptive Dual-projection Architecture for Perceptive Traversal

Dear Editor, Associate Editor, and Reviewers,

Thank you for your careful assessment and constructive suggestions. We have revised the manuscript to clarify its efficiency comparisons and adaptive sensing mechanism, explain the adaptive reward, and improve the presentation of the simulation and hardware experiments.

The principal revisions are summarized below:

- We have added deployment latency and resource-usage benchmarks against Gallant on the same local computer (Table IV(b), Sec. IV-C-3), showing 29.6% lower mean perception-to-command-publication latency and 58.4% lower sampled peak GPU memory usage, together with lower CPU utilization and RAM usage.
- We have specified the Gallant comparison: 98.6% fewer per-frame pre-encoder exteroceptive values, approximately 56% lower peak GPU memory, and 32.7% of Gallant's training-iteration time under the stated training settings.
- We have clarified that adaptive perception uses radius smoothness and regularization rather than a speed- or obstacle-dependent target radius, and specified the reward formulas and weights in the detailed responses.
- We have clarified that the adaptive radius changes only the vertical distance map; the horizontal elevation map retains fixed coverage and sampling resolution.
- We have specified that the adaptive-radius ablation in Table V is a simulation experiment, identified which two reward terms are removed, and removed the Table V comparison from Sec. IV-C-1.
- We have revised Fig. 1 and Sec. III-C to distinguish the actor from the privileged critic and show separate recurrent LSTM states. We have added error bars showing the sample standard deviation across ten independent training seeds in Fig. 5 (Sec. IV-C-2), and corrected the typo in Sec. I.

In the detailed response below, the Associate Editor's points are summarized and the reviewers' comments are reproduced with minor typographic normalization in blue italics; our responses appear in black. Each response identifies the relevant manuscript sections and figures or tables; clarifications provided only in this letter are explicitly distinguished from manuscript changes.

Response to the Associate Editor's Report (ReportOn26-3482v1)

AE Overall assessment

> *Both reviewers find the problem relevant and well motivated, and recognize the interest of adjusting how far ahead the robot perceives obstacles for different 3D locomotion tasks. The report summarizes ADAPT's horizontal and vertical projections, policy-controlled sensing radius, simulation comparisons with PIM, Humanoid Parkour, TSDF, and Gallant, and zero-shot deployment on a Unitree G1.*

Response. We sincerely thank the Associate Editor for the thoughtful assessment and for recognizing the motivation and practical relevance of the work. We also appreciate the careful summary of the dual-projection representation, adaptive sensing radius, baseline evaluations, and real-world deployment. The specific concerns raised in the report have helped us clarify the scope and evidence of our efficiency claims and improve the presentation of the method and experiments.

AE Comment 1: Evidence for compactness and computational efficiency

> *Authors claim that ADAPT provides a more compact representation and potentially makes reinforcement learning more computationally efficient, compared with voxel-based alternatives. The authors should specify how the comparisons were conducted. They have to provide enough detail to verify both the dimensionality reduction and the training-speed comparison. In particular, R8 asks where the 56% computational overhead reduction and the 98.6% observation-dimensionality reduction are supported. R3 notes that Table IV reports only training efficiency, whereas inference efficiency matters more for real-time perceptive control. R3 also asks for latency and onboard resource measurements against the main baselines.*

Response. Thank you for requesting this clarification. We agree that the efficiency metrics should be stated explicitly. The 56% figure refers to *peak GPU memory during training* relative to the Gallant voxel baseline. Using the same modified NVIDIA RTX 4090 GPU with 48 GB of VRAM and identical PPO settings (4,096 parallel environments, a rollout of 24 steps per environment per policy update, 5 PPO epochs per update, and 4 mini-batches per epoch; each rollout batch contains $4{,}096 \times 24 = 98{,}304$ transitions), peak memory is 36,054 MiB for Gallant and 15,870 MiB for ADAPT: $(36{,}054 - 15{,}870)/36{,}054 = 55.98\%$. The respective iteration times are 34.6 s and 11.3 s, so ADAPT takes 32.7% of Gallant's time under this setup.

The 98.6% figure compares *single-frame external perception inputs before encoding*. Gallant's $32 \times 32 \times 40$ voxel grid has 40,960 values; ADAPT's 21×17 horizontal elevation and 13×17 vertical distance maps have 578 values in total. Thus $(40{,}960 - 578)/40{,}960 = 98.59\%$.

To evaluate deployment efficiency, we added measurements of deployment latency and computational resource usage. Table IV(b) compares ADAPT with the main voxel baseline, Gallant, on the same local computer (Intel i7-12700H CPU and NVIDIA RTX 3060 Laptop GPU). This local computer was also used for perception and policy inference in the real-world robot experiments; we have corrected the deployment description in Sec. IV-A-2. Mean map-construction latency decreases from 64.06 to 41.64 ms, policy GPU inference latency from 1.91 to 1.40 ms, and perception-to-command-publication latency from 79.45 to 55.94 ms (29.6% lower). Sampled peak GPU memory decreases from 5,711 to 2,373 MiB (58.4% lower); mean whole-host CPU utilization decreases from 38.19% to 17.76%, and RAM usage from 10.66 to 8.04 GiB.

Changes in the revised manuscript.

- Sec. I restates the two efficiency quantities.

- Table I in Sec. II-A defines the pre-encoder sensory dimension; Sec. IV-B-3 derives the comparison from these dimensions and the training measurements in Table IV(a).
- Table IV(b) and Sec. IV-C-3 report the local-computer runtime comparison, measurement boundaries, hardware, and trial setup.

Location in the revised manuscript: Abstract and Sec. I, p. 1; Sec. II-A and Table I, p. 2; Sec. IV-B-3, p. 5; Table IV(a)–(b), p. 6; Sec. IV-C-3, p. 7.

AE Comment 2: Voxel baselines and adaptive reward

> *Voxel-based representations are not inherently inferior and can have different computational trade-offs. Analyze whether the voxel-based baselines' performance gap arises from optimization difficulty, network capacity, training budget, representation redundancy, or implementation factors. Explain the role of the adaptive reward in learning the radius and clarify whether it encodes a hand-designed heuristic.*

Response. Thank you for this helpful suggestion. We agree that the baseline results warrant further analysis. We have added an analysis in Sec. IV-B-3 (Performance Comparison). Voxel representations retain richer 3D geometry and are not inherently inferior. For the Gallant voxel baseline implemented in this study, however, fine spatial resolution limits sensing coverage under a fixed voxel count and computational budget. The success-rate gap relative to ADAPT is smaller at lower speeds and widens at high speed. In high-speed tests, we observed abrupt deceleration after obstacles entered the sensing range, sometimes followed by loss of balance. We attribute this behavior to limited look-ahead, which may leave insufficient time for smooth adjustments of speed and posture and thus increase the difficulty of high-speed traversal. In contrast, ADAPT allows the policy to adjust its sensing range, and hence its look-ahead distance, according to the robot's motion state.

The adaptive reward contains radius smoothness and regularization terms. With current radius r_t, previous radius r_{t-1}, and initial radius r_{init}, their contributions are $R_{\text{smooth}}(t) = -\lambda_s|r_t - r_{t-1}|$ and $R_{\text{reg}}(t) = -\lambda_r(r_t - r_{\text{init}})^2$, where $\lambda_s = \lambda_r = 0.1$ during training. Both terms are removed in the reward ablation, while the radius remains in the observation and action spaces. The terms discourage abrupt radius changes and drift from the initial radius; neither prescribes a target radius based on speed or obstacle distance. Our interpretation is that, without these terms, unstable radius choices can make future observations less consistent, especially at higher speeds. The ablation removes both terms together, so it does not isolate either term's causal contribution.

Changes in the revised manuscript.

- Sec. IV-B-3 adds the voxel resolution–coverage trade-off, the speed-dependent performance gap, and the observed abrupt deceleration and loss of balance, together with the limited-look-ahead explanation.
- The Active Perception item in Sec. III-B-3 describes the radius smoothness and initial-radius deviation penalties.
- Sec. IV-B-4 specifies that the reward ablation removes both terms while retaining the radius in the observation and action spaces, and discusses their role in stabilizing radius selection at higher speeds (Table V).

Location in the revised manuscript: Sec. III-B-3, pp. 3–4; Secs. IV-B-3–IV-B-4, pp. 5–6; Tables III and V, p. 6.

AE Comment 3: Clarifications and presentation

Resolve whether the radius changes the vertical map only or both maps. State whether the adaptive-radius ablation was run in simulation or on hardware. Clarify whether the robot recovers autonomously in the drop experiment or the safety tether arrests the fall. Add confidence intervals or variance to Fig. 5, and correct the likely typo in the Introduction.

Response. Thank you for these constructive comments. We agree that these details should be clarified. The radius affects only the vertical distance map; horizontal-map coverage and resolution are fixed. The Table V adaptive-radius ablation was run in simulation. In the repeated drop experiment, the robot recovered autonomously; the tether was a hardware safety measure and did not arrest the fall. We reduced its tension and repeated the experiment ten times in the same scene with the same policy to clarify its role. We have replaced the drop-experiment video clip in the Multimedia Materials to make the role of the safety tether clearer. We revised Fig. 5 to show the mean and sample standard deviation of success rates across ten independently trained policies, each evaluated using its final checkpoint under the same protocol. We also removed the erroneous wording *3-D negotiation tasks* from the Introduction.

Changes in the revised manuscript.

- We clarified the maps in Sec. I, Sec. II-B, and Sec. III-B-1.
- We identified the simulation ablation setting in Sec. IV-B-4 and removed the Table V comparison from Sec. IV-C-1.
- We revised Fig. 5 and its caption in Sec. IV-C-2.
- We corrected the typo in Sec. I.

Location in the revised manuscript: Sec. I, p. 1; Sec. II-B, p. 2; Sec. III-B-1, p. 3; Sec. IV-B-4, pp. 5–6; Table V, p. 6; Sec. IV-C-1, pp. 6–7; Sec. IV-C-2 and Fig. 5, p. 7. Tether clarification: response letter and updated drop-experiment video clip in the Multimedia Materials.

Response to Reviewer 3

Reviewer 3, General assessment

> *This paper presents ADAPT, a perceptive locomotion framework for humanoid robots that decomposes local 3D geometry into two projections: a horizontal elevation map and a vertical distance map. The proposed approach is evaluated against several representative baselines and is further deployed on a Unitree G1 humanoid robot in the real world.*

Response. Thank you for your careful reading and for recognizing both the comparative evaluation and the real-world Unitree G1 deployment. We appreciate your constructive comments, which have helped us make the method, baseline comparisons, and deployment evidence clearer.

Reviewer 3, Comment 1: Motivation

> *This paper addresses an important problem in perceptive locomotion, and the motivation is well established.*

Response. Thank you for recognizing the importance and motivation of this work. ADAPT aims to coordinate limited perceptual resolution with the different look-ahead requirements of locomotion in complex three-dimensional environments.

Reviewer 3, Comment 2: Adaptive perception radius

> *Incorporating the perception radius into the policy's action space is a clever design choice. It allows the robot to adaptively adjust its perceptual range according to the requirements of different 3D locomotion tasks.*

Response. Thank you for this assessment. Including the radius in the action space lets the policy learn locomotion and sensing-range adjustment together. The vertical map's input dimension stays fixed while its radial coverage and sampling interval vary.

Reviewer 3, Comment 3: Voxel-based baseline analysis

> *Table III shows that ADAPT outperforms all baselines, including some computationally intensive voxel-based approaches. Given that voxel representations retain richer 3D geometric information, it would be helpful for the authors to provide further analysis explaining why these methods perform worse than ADAPT. In particular, the paper should clarify whether the performance gap arises from optimization difficulty, limited network capacity, training-budget constraints, representation redundancy, or other implementation factors.*

Response. Thank you for this suggestion. We have added an analysis in Sec. IV-B-3 (Performance Comparison). Voxel representations retain richer 3D geometry and are not inherently inferior. For the Gallant voxel baseline implemented in this study, however, fine spatial resolution limits sensing coverage under a fixed voxel count and computational budget. The success-rate gap relative to ADAPT is smaller at lower speeds and widens at high speed. In high-speed tests, we observed abrupt deceleration after obstacles entered the sensing range, sometimes followed by loss of balance. We attribute this behavior to limited look-ahead, which may leave insufficient time for smooth adjustments of speed and posture and thus increase the difficulty of high-speed traversal. In contrast, ADAPT allows the policy to adjust its sensing range, and hence its look-ahead distance, according to the robot's motion state.

Changes in the revised manuscript.

- Sec. IV-B-3 now explains that, for the implemented Gallant baseline, fine voxel resolution restricts sensing coverage under a fixed voxel count and computational budget.
- The same section describes the smaller performance gap at lower speeds and the abrupt deceleration and occasional loss of balance observed at high speed, explaining how limited look-ahead can hinder smooth speed and posture adjustments.
- The analysis acknowledges that voxels preserve richer 3D geometry and contrasts the implemented Gallant baseline's fixed sensing coverage with ADAPT's policy-controlled look-ahead distance.

Location in the revised manuscript: Sec. IV-B-3, p. 5; Table III, p. 6.

Reviewer 3, Comment 4: Drop experiment and tether

> *The real-world experiments provide valuable hardware validation. However, for the drop experiment, it is difficult to determine whether the robot recovers its balance autonomously or whether its fall is arrested by the safety tether.*

Response. Thank you for pointing out the video ambiguity. In our repeated drop experiment, the robot recovered autonomously; the tether served as a hardware safety measure rather than arresting the fall. We reduced its tension and repeated the experiment ten times in the same scene with the same policy to clarify its role. We have replaced the drop-experiment video clip in the Multimedia Materials to make the role of the safety tether clearer.

Changes in the revised manuscript. We have replaced the drop-experiment video clip in the Multimedia Materials.

Location in the revised manuscript: Response letter and Multimedia Materials (updated drop-experiment video clip); no manuscript text change.

Reviewer 3, Comment 5: Deployment latency and onboard resources

> *Table IV reports training efficiency, but inference efficiency is arguably more important for perceptive locomotion because training is performed offline, whereas perception and control must operate under strict real-time constraints during deployment. The authors are encouraged to report inference-time benchmarks, including map-construction latency, policy inference latency, total perception-to-control latency, and onboard computational resource usage. A comparison of these quantities across ADAPT and the main baselines would better support the paper's efficiency claims.*

Response. Thank you for this suggestion. We have extended Table IV with runtime measurements for ADAPT and the Gallant voxel baseline and added the measurement setup and analysis in Sec. IV-C-3. Both methods were deployed and benchmarked on the same local computer with an Intel i7-12700H CPU and an NVIDIA RTX 3060 Laptop GPU with 6 GiB VRAM. Both the real-world robot experiments and the latency and resource-usage benchmarks used this local-computer deployment setup. Perception and policy inference ran on the local computer, which generated and sent joint-angle targets to the robot at 50 Hz. No Jetson Orin NX was used in either set of experiments. We have corrected the original deployment description in Sec. IV-A-2 and clarified the shared setup in Sec. IV-C-3. Table IV(b) reports latency and resource usage on the local computer used for deployment.

Each method was tested five times on each of three terrain categories: flat ground, a single obstacle, and a complex environment, for 15 trials per method. Latency entries are equal-weight averages of per-trial means

and P95s. Map latency spans the input callback to the first observed map-ready event; perception-to-publish latency spans that callback to the first command-publication call using the update. Policy GPU inference latency is measured with current-stream CUDA events. For ADAPT, each map-latency and perception-to-publish entry reports the larger statistic of its two map streams. CPU utilization and RAM usage are whole-host averages, and peak VRAM is the maximum sampled usage.

Relative to Gallant, ADAPT reduces mean map latency from 64.06 to 41.64 ms and mean perception-to-publish latency from 79.45 to 55.94 ms, a 29.6% reduction for the latter. Mean policy GPU inference latency decreases from 1.91 to 1.40 ms, and sampled peak VRAM decreases from 5,711 to 2,373 MiB (58.4% lower). Mean CPU utilization decreases from 38.19% to 17.76%, and RAM usage from 10.66 to 8.04 GiB.

Changes in the revised manuscript. Sec. IV-A-2 now states that real-world perception and policy inference run on the local computer, which sends joint-angle targets to the robot at 50 Hz. Table IV(b) and Sec. IV-C-3 report the runtime comparison on the same deployment computer, measurement boundaries, hardware, and trial setup.

Location in the revised manuscript: Abstract and Sec. I, p. 1; Sec. IV-A-2, p. 5; Table IV(b), p. 6; Sec. IV-C-3, p. 7.

Response to Reviewer 8

Reviewer 8, General assessment

> *The paper proposes ADAPT, a perceptive locomotion framework for humanoid robots. ADAPT decomposes local 3D geometry into two compact projections: a horizontal elevation map for footholds and a vertical distance map for body-level clearance. Moreover, it adds the sensing radius as an extra action of the reinforcement learning policy. The authors compare ADAPT in simulation against PIM, Humanoid Parkour, a TSDF, and Gallant, and deployed zero-shot on a Unitree G1.*
>
> *To my knowledge, the content of the article is novel and sufficiently significant. The authors justify their method well and are clear in the explanation of the limits of baseline methods and the strengths of their proposed technique. The experiments validate the authors' assumptions. The organization of the paper is adequate, with one exception listed below. To my knowledge, the citations are adequate. I believe this to be a valuable contribution, still I believe some parts could be clearer.*

Response. We are grateful for your encouraging assessment of the novelty and significance of ADAPT and for your recognition of the motivation, comparisons, and experimental validation. Thank you also for noting that the organization and citations are generally adequate. Your detailed suggestions have helped us resolve the remaining presentation issue and explain the method and results more precisely.

Reviewer 8, Comment 1: Typo

> *There is a possible typo in the introduction: 3-D negotiation tasks, you mean 3-D navigation tasks?*

Response. Thank you for catching this error. We agree and have corrected it. We removed the erroneous wording *3-D negotiation tasks* when revising Sec. I (Introduction).

Location in the revised manuscript: Sec. I, p. 1.

Reviewer 8, Comment 2: Support for the efficiency claims

> *In the introduction, you claim that the adaptive sensing mechanism is responsible for reducing the computational overhead by 56%. Where in the experiment section do you support this claim? The same question holds for the 98.6% reduction in observation dimensionality with respect to voxel-based methods that is claimed in the abstract. I think this comparison could be more clearly stated.*

Response. Thank you for pointing this out. We agree that the original comparison was imprecise. The 56% result is the reduction in training *peak GPU memory* relative to Gallant: $(36{,}054-15{,}870)/36{,}054 = 55.98\%$, with values in MiB. Using the same modified NVIDIA RTX 4090 GPU with 48 GB of VRAM and common PPO settings (4,096 parallel environments, a rollout of 24 steps per environment per policy update, 5 PPO epochs per update, and 4 mini-batches per epoch; each rollout batch contains $4{,}096 \times 24 = 98{,}304$ transitions), iteration time is 34.6 s for Gallant and 11.3 s for ADAPT; ADAPT takes 32.7% of the baseline's time. These results compare the implemented representation and encoder pipelines, not the isolated effect of adaptive-radius selection.

The 98.6% result compares only single-frame perception inputs *before encoding*: $32 \times 32 \times 40 = 40{,}960$ voxel values for Gallant versus $21 \times 17 + 13 \times 17 = 578$ map values for ADAPT, giving $(40{,}960 - 578)/40{,}960 =$

98.59%. It does not compare total actor observations.

Changes in the revised manuscript.

- The Abstract states a 98.6% reduction in exteroceptive input dimensionality relative to our implementation of Gallant and qualitatively summarizes lower deployment latency and resource usage.
- Sec. I gives the quantitative training and local-computer deployment comparisons.
- The caption of Table I in Sec. II-A defines sensory dimension as pre-encoder input elements, excluding proprioception, history, and critic-only inputs.
- Sec. IV-B-3 gives the dimensionality calculation and common training settings; Table IV(a) reports wall-clock iteration times and peak GPU memory.

Location in the revised manuscript: Abstract and Sec. I, p. 1; Sec. II-A and Table I, p. 2; Sec. IV-B-3, p. 5; Table IV(a)–(b), p. 6; Sec. IV-C-3, p. 7.

Reviewer 8, Comment 3: Which map the radius changes

> *The abstract, introduction, and Sec. II-B say the radius modulates the vertical map only. Sec. III-B-1 says it acts as a dynamic gain modulating the field of view for both projection maps. Is the horizontal map also influenced by the radius?*

Response. Thank you for highlighting this ambiguity. We agree that the scope of radius adaptation should be explicit. The adaptive radius affects only the vertical distance map. Horizontal-map coverage and sampling resolution remain fixed.

Changes in the revised manuscript.

- Sec. I, Sec. III-A, Sec. III-B-1, Sec. III-B-2, and Sec. III-C specify that r_t changes only the vertical map's radial range and sampling interval.
- The horizontal map retains fixed coverage and sampling resolution, consistent with Sec. II-B.

Location in the revised manuscript: Abstract and Sec. I, p. 1; Secs. II-B and III-A, p. 2; Secs. III-B-1–III-B-2, p. 3; Sec. III-C and Fig. 2, p. 4.

Reviewer 8, Comment 4: Ablation setting

> *Was the ablation study on the adaptive radius mechanism done in simulation or in the real-world? Discussing Table V results in Sec. IV-C-1 creates confusion.*

Response. Thank you for requesting this clarification. We agree that the evaluation setting should be explicit. The adaptive-radius ablation in Table V was conducted in simulation.

Changes in the revised manuscript.

- Sec. IV-B-4 identifies the tests as simulation experiments, with ten trials per method, terrain, and speed command, and specifies the two removed radius-reward terms.
- The same section clarifies that each trial contains five obstacles and that success is the fraction of obstacles crossed, with statistics pooled across the three terrains at each speed.
- We removed the Table V comparison from Sec. IV-C-1, which covers hardware tests.

Location in the revised manuscript: Sec. IV-B-4, pp. 5–6; Table V, p. 6; Sec. IV-C-1, pp. 6–7.

Reviewer 8, Comment 5: Adaptive reward

> *The ablation shows that without the adaptive reward the policy fails to learn the optimal radius as speed increases. Do you have an explanation on why that happens? Which part of the adaptive reward is responsible for retrieving the optimal policy? Is the optimal reward basically enforcing a hand-designed heuristic? Understanding this could help better define the scope of the contribution.*

Response. Thank you for this helpful comment. We agree that the reward formulation needs a clearer explanation. The training reward for adaptive perception contains a smoothness term and a radius regularizer. With current radius r_t, previous radius r_{t-1}, and initial radius r_{init}, their contributions to the total reward can be written as

$$R_{\text{smooth}}(t) = -\lambda_s |r_t - r_{t-1}|, \qquad R_{\text{reg}}(t) = -\lambda_r (r_t - r_{\text{init}})^2,$$

where $\lambda_s = \lambda_r = 0.1$ during training. Thus, the combined radius-related contribution to the total reward is $-0.1|r_t - r_{t-1}| - 0.1(r_t - r_{\text{init}})^2$. Both contributions are removed in the reward ablation, while the radius remains in the observation and action spaces and all other reward terms are retained. The terms penalize abrupt radius changes and drift from the initial radius. Neither sets a target radius from speed or obstacle distance, and training uses no target-radius tracking reward. The regularizer encodes a preference for the initial radius, while allowing task rewards to favor a different radius when beneficial; it does not prescribe a speed-to-radius rule.

Our interpretation is that radius actions affect locomotion indirectly through future perception, which may be harder to learn than direct motor actions. Without regularization, early radius oscillation or drift can make observations less consistent; higher speeds may increase sensitivity to unsuitable or unstable sensing ranges. This is an interpretation of the ablation, not a separate causal measurement of either reward term.

Changes in the revised manuscript.

- The Active Perception item in Sec. III-B-3 describes the smoothness and initial-radius deviation penalties.
- Sec. IV-B-4 states that the reward ablation removes both terms and interprets the high-speed result.
- The exact formulas, training weights, and clarification that no speed- or obstacle-dependent target radius is prescribed are provided directly in this response.

Location in the revised manuscript: Sec. III-B-3, pp. 3–4; Sec. IV-B-4, pp. 5–6; Table V, p. 6. Formulas and weights: response letter.

Reviewer 8, Comment 6: Actor inputs and recurrence

> *Fig. 1 makes it look like the actor also receives the privileged states, which contradicts what is claimed by the text in Sec. III-C. I also think Fig. 1 could be more informative if it showed the recurrent nature of the actor.*

Response. Thank you for identifying the diagram error. We revised Fig. 1 so that the actor receives only deployable observations while the separate critic receives the additional privileged elevation map. The figure now shows independent actor and critic LSTM hidden-state loops. Each branch uses an LSTM with a hidden dimension of 256, followed by an MLP with hidden dimensions $[128, 64, 32]$ and ELU activations. The horizontal-map MLP encoder has hidden dimensions $[256, 128, 64]$, and the vertical-map MLP encoder has hidden dimensions $[128, 64]$; both use CELU activations and produce 32-dimensional embeddings. The two

map embeddings form a 64-dimensional environmental representation. The actor and critic maintain separate recurrent states, and the privileged ground-truth elevation map is supplied only to the critic.

Changes in the revised manuscript. Fig. 1 in Sec. III-B-1 (Observation Space) and Sec. III-C (Training Architecture) now distinguish the inputs and recurrent states of the actor and critic.

Location in the revised manuscript: Sec. III-B-1 and Fig. 1, p. 3; Sec. III-C, p. 4. Layer specifications: response letter.

Reviewer 8, Comment 7: Uncertainty in Fig. 5

> *Fig. 5 only shows the average success rates, it would be better to show also confidence intervals/variance. The plot as it is does not give information on how much performance varies across seeds.*

Response. Thank you for highlighting the variability across seeds. The experiments use ten policies independently trained with different random seeds, with the final checkpoint from each training run evaluated under the same protocol. For each terrain in each domain (simulation and real world), each policy is tested once at each of three command speeds (0.5, 1.0, and 1.5 m/s), giving 30 binary outcomes. We first average the three outcomes for each policy, then report the mean and sample standard deviation of these ten policy-level success rates in Fig. 5. Thus, the error bars show variability across independently trained policies under this evaluation protocol; they are not confidence intervals computed by treating all 30 outcomes as independent. Because each policy is tested only once per speed, the measured variability can also reflect evaluation randomness. The simulation evaluation uses single-obstacle configurations matched to the fixed real-world scenes, distinct from the five-obstacle tracks in Table III. Success means traversing the single obstacle.

Changes in the revised manuscript.

- Sec. IV-C-1 specifies ten independent training seeds, final-checkpoint selection, and the common evaluation protocol: one trial per policy at each of three speeds, for 30 trials per terrain and domain.
- Fig. 5 and its caption in Sec. IV-C-2 now report the mean and sample standard deviation across the ten policy-level success rates and specify single-obstacle evaluation in matched simulation and real-world scenes.

Location in the revised manuscript: Secs. IV-C-1–IV-C-2, pp. 6–7; Fig. 5, p. 7.

ADAPT: Adaptive Dual-projection Architecture for Perceptive Traversal

***Abstract*— Agile humanoid locomotion in complex 3D environments requires reasoning about both ground-support geometry and body-level traversable space under the strict latency constraints of high-frequency control. Existing perceptive locomotion methods often face two coupled limitations: compact 2.5D maps are efficient but miss vertical clearance constraints, whereas dense 3D representations provide richer geometry at higher computational cost; meanwhile, fixed sensing scale cannot adapt to changes in motion speed and local environmental complexity. We propose ADAPT (Adaptive Dual-projection Architecture for Perceptive Traversal), a perception-control framework that addresses both representation efficiency and sensing-scale adaptation. ADAPT decomposes local 3D geometry into two compact projections: a horizontal elevation map for terrain support and a vertical distance map for body-level clearance. It further treats the sensing radius of the vertical map as a reinforcement learning policy action, enabling the robot to expand its perceptual horizon for long-range anticipation and contract it for higher local resolution in cluttered spaces. Experiments show that ADAPT achieves a 94.75% success rate across diverse 3D traversal tasks, and reduces the exteroceptive observation input dimensionality by 98.6% relative to our implementation of the Gallant baseline, while also lowering deployment latency and computational resource usage. The learned policy achieves zero-shot transfer to a Unitree G1 humanoid robot and enables real-world space traversal.**

I. Introduction

Humanoid robots are expected to move through human-scale environments that contain stairs, cluttered passages, protruding structures and other three-dimensional constraints. For successful traversal, the controller depends on not only estimating where the feet can step on, but also understanding whether the whole body can safely pass through the surrounding space, including overhead and lateral obstacles that may cause collision or loss of balance. This makes perceptive traversal a coupled problem of terrain understanding, body-level clearance reasoning and real-time whole-body control.

However, onboard perception for legged robots is constrained by strict latency and bandwidth requirements. Existing representations therefore tend to emphasize either efficiency or geometric completeness. Conventional 2.5D elevation maps [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11] provide compact terrain geometry for foothold reasoning, but they are intrinsically blind to overhangs, side walls and body-level clearance constraints. Depth-based representations [12], [13], [14], [15], [16], [17], [18], [19], [20] offer direct visual observations, yet they can be affected by view changes, occlusion and illumination variation. Volumetric approaches [21], [22], [23] encode richer 3D geometry, but their dimensionality and processing cost make high-frequency policy learning and deployment difficult. These limitations necessitate a representation that preserves 3D traversability while remaining computationally compact [24], [25], [26].

A second limitation is that perception is usually acquired with a fixed spatial range [21], [9], [4], which is misaligned with the changing information demand of locomotion: fast motion needs a longer horizon while cluttered traversal needs higher local resolution. A perceptive controller should therefore decide not only how to act given perception, but also how much spatial context should be sensed.

To address these two issues, this paper proposes **ADAPT** (**A**daptive **D**ual-projection **A**rchitecture for **P**erceptive **T**raversal) that restructures local 3D geometry into two orthogonal and information-dense projections. A horizontal elevation map describes ground geometry for stable foothold selection, while a vertical distance map describes body-level traversable space for avoiding overhangs and obstacles. Instead of using a fixed perceptual horizon, ADAPT further exposes the sensing radius of the vertical distance map to the reinforcement learning policy as a control variable. The horizontal elevation map retains a fixed spatial extent and sampling resolution. The policy can thus expand the vertical map's perceptual horizon during fast traversal and contract it for fine-grained local reasoning in cluttered scenes. In this way, ADAPT jointly addresses what spatial information should be represented and how much of it should be sensed at each moment.

Empirical evaluation shows that ADAPT achieves a mean success rate of 94.75% across eight terrain categories and reduces per-frame actor exteroceptive dimensionality by 98.6% relative to Gallant. Under matched training settings, it uses 56% less peak GPU memory and 32.7% of Gallant's iteration time. On the same local computer, perception-to-command-publication latency and sampled peak GPU memory decrease by 29.6% and 58.4%, respectively (Section IV).

The main contributions of this work are:

- Proposing a highly efficient formulation that decouples complex 3D environments into compact projections. This paradigm reduces observational dimensionality while rigorously preserving essential geometric constraints, thereby ensuring the high-frequency perception strictly required for agile locomotion.
- Introducing a bidirectional closed-loop mechanism that embeds perceptual configurations directly into the reinforcement learning action space. This formulation empowers the robot to autonomously modulate its sensory settings based on instantaneous dynamic states and environmental complexity, thereby achieving an optimal trade-off between navigational safety and precise spatial

perception.
- Demonstrating robust zero-shot sim-to-real transfer on a Unitree G1 humanoid robot with successful navigation through diverse structural challenges.

II. Related Works

A. Perception for Legged Locomotion

Table I compares elevation, image/depth, and volumetric or point-based representations in terms of geometry, input size, and sensing adaptation.

TABLE I: ADAPT comparison with existing works. Sensory dimension counts the per-frame perceptual input elements before encoding, excluding proprioception, observation history, and critic-only privileged inputs.

Method	Perceptual Representation	Native Sampling	Sensory Dimension	Overhead	Adaptive Sensing
PIM [4]	Elevation Map	0.10 m	96	✗	✗
Beamdojo [5]	Elevation Map	0.10 m	225	✗	✗
Vb-com [6]	Elevation Map	0.10 m	225	✗	✗
Parkour [14]	Depth Image	48×64 px	3072	✓	✗
Omni [23]	Point Cloud	0.05π rad	251	✓	✗
Gallant [21]	Voxel Grid	0.05 m	40960	✓	✗
HumanoidPF [27]	Joint Position	39 dims	39	✓	✗
ADAPT (ours)	Dual Projection	**0.025π rad**	**578**	✓	✓

Elevation maps [1], [2], [4], [5], [6], [7], [11] provide efficient 2.5D grids for foothold reasoning. However, their height-field assumption inherently discards vertical structures like overhangs and side walls, missing the body-level clearance constraints crucial for humanoid traversal in 3D clutter.

Depth and vision-based methods [13], [12], [14], [15], [16], [18], [19], [20] offer richer scene observations for agile locomotion. Yet, their unstructured inputs heavily burden the learning system and are susceptible to occlusions, viewpoint shifts, and illumination changes. While reduced visual encodings mitigate this, they tend to entangle ground geometry with visual noise rather than explicitly separating the needed spatial constraints.

Volumetric, point-cloud and multi-plane representations aim to recover more complete 3D spatial awareness [9], [21], [22], [23], [27]. These methods are better suited to collision avoidance and constrained-space traversal because they can encode obstacles beyond the terrain surface. Their main drawback is the cost of maintaining and processing dense 3D structure. Voxel grids scale cubically with resolution and incur high memory traffic [24], leading to severe inference latency (e.g., 4.4 Hz in [25]) that fails the ≥ 10 Hz requirement for closed-loop control [26] and inflated training iteration times (e.g., $>$ 22 s/iter in [21]). Point-based representations require careful sampling and aggregation to remain stable for policy learning. Multi-plane methods mitigate the cubic cost but leave vertical clearance implicit, forcing the policy to infer the gap between an overhead obstacle and the supporting ground across plane indices rather than from a direct distance measurement. The increased latency and observation dimensionality can slow training and reduce the responsiveness needed for high-speed humanoid motion.

ADAPT bridges these approaches with two compact projections that explicitly separate ground support from body-level clearance without encoding the full local volume.

B. Adaptive Perception and Behavior Modulation

The need for adaptive perception arises from the non-uniform information demand of locomotion: fast traversal requires a long perceptual horizon to anticipate obstacles, whereas cluttered traversal benefits more from high-resolution local geometry. A fixed sensing range cannot satisfy both simultaneously.

Spatial attention and foveation methods select informative regions or suppress irrelevant visual content [28], [29], while multi-range mapping assigns different densities to nearby and distant regions [30]. However, these mechanisms adapt perception after the sensing structure has been specified; the spatial extent and resolution levels remain fixed by design rather than continuously selected by the policy.

Active perception mechanisms change the sensing focus or viewpoint according to task context [31], and perceptive locomotion methods adapt gait and footholds based on terrain observations [14], [4], [5], [6]. Nevertheless, perception is typically organized as an upstream module with a fixed range, and the downstream policy adapts actions to those observations. The missing component is a feedback path in which the same policy that generates motion also regulates the amount of spatial information it receives.

ADAPT closes this feedback loop by making only the vertical map's sensing radius a policy action and including the previous radius in the observation. The horizontal map retains fixed coverage and resolution. Reinforcement learning jointly adapts locomotion and sensing range without changing the sensor pose.

III. Methods

A. Problem Formulation

The perceptive traversal task is formalized as a goal-conditioned Partially Observable Markov Decision Process (POMDP). At step t, policy π_θ receives the robot state and local geometry in $\mathbf{o}_t$ and outputs joint targets and a vertical-map sensing radius, $\mathbf{a}_t = (\mathbf{q}_t, r_t)$.

Driven by the stochastic transition dynamics $\mathcal{T}$, the agent seeks to learn an optimal policy π^* that maximizes the expected cumulative discounted return:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t \mathcal{R}(s_t, a_t) \right], \tag{1}$$

where $\gamma \in (0, 1)$ is the discount factor and T is the horizon. The scalar reward $\mathcal{R}(s_t, a_t)$ encodes multiple objectives, including velocity tracking, energy efficiency, and safety constraints.

B. Training Pipeline

The training pipeline is illustrated in Fig. 1.

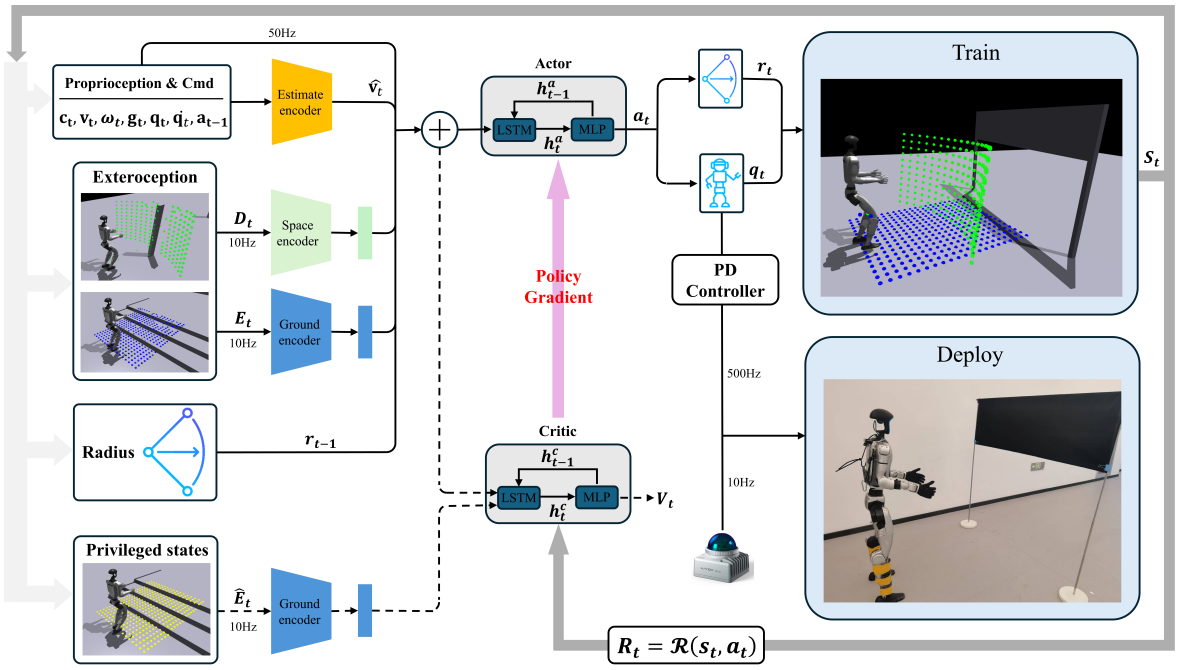

Fig. 1: Illustration of the training architecture for the proposed control framework ADAPT.

1) Observation Space: The observation vector o_t integrates proprioceptive feedback, task commands and exteroceptive data, which is defined as

$$\mathbf{o}_t = \{\mathbf{c}_t, \mathbf{v}_t, \boldsymbol{\omega}_t, \mathbf{g}_t, \mathbf{q}_t, \dot{\mathbf{q}}_t, \mathbf{a}_{t-1}, \mathbf{E}_t, \mathbf{D}_t, r_{t-1}\}. \quad (2)$$

The specific components in (2) are:

- *Task Command:* The velocity command $\mathbf{c}_t \in \mathbb{R}^3$.
- *Proprioceptive States:* These states include base linear velocity $\mathbf{v}_t$, base angular velocity $\boldsymbol{\omega}_t$, the gravitational acceleration $\mathbf{g}_t$, joint positions $\mathbf{q}_t$, joint velocities $\dot{\mathbf{q}}_t$ and the preceding action $\mathbf{a}_{t-1}$.
- *Exteroceptive States:* During learning, synthetic point clouds are acquired via the LiDAR simulation framework from Wang et al. [23]. We rasterize the raw point cloud into grid structures used by both the actor and critic to facilitate efficient processing. Consequently, this input is processed into two orthogonal robot-centric maps as shown in Fig. 1: (1) A *horizontal elevation map* ($\mathbf{E}_t \in \mathbb{R}^{21\times17}$) that captures terrain geometry via discrete height sampling. (2) A *vertical distance map* ($\mathbf{D}_t \in \mathbb{R}^{13\times17}$) that encodes spatial traversability. The distance map is generated by projecting points into a cylindrical sector spanning from -45 to $+45$ degrees around the vertical axis, where each point records the distance to the nearest obstacle.
- *Adaptive Perception Radius:* The radius r_{t-1} controls only the radial sensing range of the vertical distance map $\mathbf{D}_t$. The horizontal elevation map $\mathbf{E}_t$ uses a fixed spatial extent and sampling resolution, both independent of this radius. It is constrained to $r_t \in [1, 5]$ m throughout training and deployment. The vertical distance map uses a cylindrical sector grid with a fixed number of bins $(N_r, N_\theta) = (13, 17)$ as illustrated in Fig. 2. For a selected radius r_{t-1}, the radial sampling interval is defined as $\delta_r = r_{t-1}/N_r$, which makes the radial sampling interval of the vertical distance map grow linearly with the sensing radius; the angular span and number of bins remain fixed. Therefore, a larger radius extends the perceptual horizon but coarsens sampling resolution, while a smaller radius concentrates the grids and yields finer resolution. When long-range perception is unnecessary, reducing the sensing radius can also limit the influence of distal sensor noise by excluding distant returns from the map.

The critic operates on a privileged observation space to facilitate accurate estimation of the value function V_t. In addition to the actor's input vector o_t, the critic receives the noise-free ground-truth elevation map $\hat{\mathbf{E}}_t$ directly acquired from the simulation engine.

2) Action Space: At 50 Hz, the policy outputs $\mu_t \in \mathbb{R}^{13}$. Its first 12 elements set joint targets for a low-level Proportional-Derivative (PD) controller:

$$q_t = s_a \cdot \mu_{t,1:12} + q_{\text{default}}, \quad (3)$$

where s_a is the action scale and q_{default} the nominal configuration. *Adaptive Perception Radius:* The final element sets $r_t \in [1, 5]$ m, adjusting only the next vertical distance map's radial range and sampling interval. This empirical range avoids insufficient lookahead below 1 m and excessive distal noise above 5 m.

3) Reward Functions: The reward function $\mathcal{R}$ integrates multiple objectives to encourage robust and efficient locomotion adhering to safety constraints. The total reward $\mathcal{R}_t = \sum_i w_i r_i(t)$ is a weighted sum of the following components:

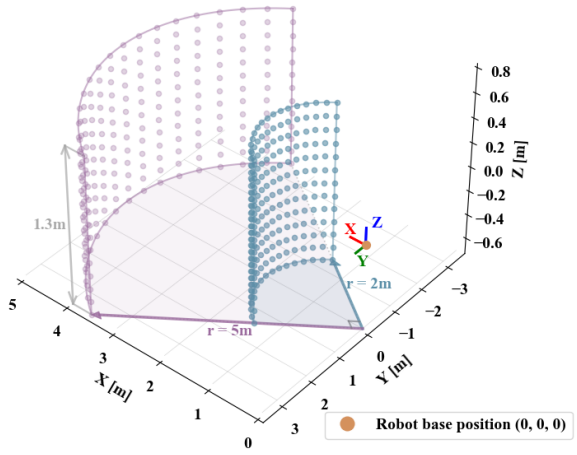

Fig. 2: Illustration of the adaptive sensing radius mechanism for the vertical distance map. The vertical distance map is constructed on a cylindrical sector with a fixed number of cells (17×13). A smaller radius (blue) provides higher spatial resolution for precise near-field sensing, while a larger radius (purple) extends the perception range at the cost of reduced resolution. The policy learns to dynamically adjust this radius based on the robot's locomotion state.

- *Motion Control:* Precise velocity tracking, postural stability, and energy efficiency are promoted by penalizing deviations in base orientation, excessive joint velocities or accelerations, and high-frequency action oscillations.
- *Safety Mechanisms:* Safe and prolonged operation is ensured by penalizing critical failures such as body collisions, mesh interpenetrations, and joint limit violations. Furthermore, survival bonuses and foot airtime rewards are provided to foster dynamic and robust obstacle clearance.
- *Active Perception:* The sensing radius of the vertical distance map is learned jointly with locomotion via the task rewards and two radius regularization terms: a smoothness term penalizes frame-to-frame radius changes, and a deviation term penalizes squared deviations from the initial radius, ensuring smooth and stable sensing adjustments.

4) Terrain Design: BarrierTrack (Fig. 3), a customized environment assembling discrete obstacle modules, is designed to cultivate versatile locomotion. Each episode features a sequence of five obstacles of the same terrain type with progressively increasing difficulty according to a curriculum schedule.

A curriculum-based strategy is adopted where each terrain type τ is parameterized by a difficulty scalar $s \in [0, 1]$, and its generation parameters are interpolated as $\mathbf{p}_\tau(s) = (1 - s)\,\mathbf{p}_\tau^{\min} + s\,\mathbf{p}_\tau^{\max}$ based on Table II. Perlin noise is superimposed onto ground surfaces to simulate terrain irregularity and prevent overfitting.

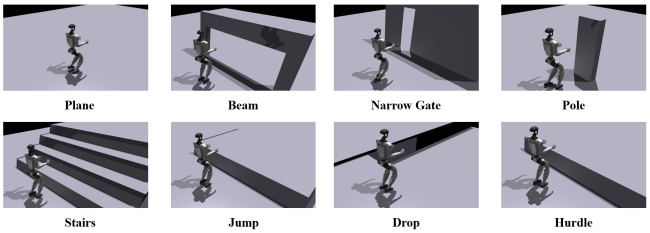

Fig. 3: Examples of procedurally generated obstacles in our *BarrierTrack* training environment. The curriculum includes a diverse set of challenges such as stairs, drop, hurdles, and constrained passages like beams and narrow gates, promoting the development of a versatile locomotion policy.

TABLE II: Parameters for generating curriculum training terrains. The ↑ symbol indicates the parameter value increases as difficulty scales up ($\mathbf{p}_\tau^{\min} < \mathbf{p}_\tau^{\max}$), while the ↓ symbol indicates the value decreases as the task becomes harder ($\mathbf{p}_\tau^{\min} > \mathbf{p}_\tau^{\max}$).

Terrain	Param. (m)	$\mathbf{p}_\tau^{\min}$	$\mathbf{p}_\tau^{\max}$
Beam	Clearance Height ↓	1.33	1.10
	Beam Depth ↑	0.20	0.20
Narrow Gate	Gate Width ↓	0.80	0.45
	Position Offset ↑	0.00	0.60
Pole	Pole Width ↓	0.50	0.10
	Position Offset ↑	0.00	0.00
Stairs	Step Height ↑	0.10	0.30
	Step Length ↓	0.50	0.30
Jump	Wall Height ↑	0.05	0.40
	Wall Depth ↑	0.10	0.30
Drop	Drop Height ↑	0.10	0.60
	Platform Length ↓	0.50	0.30
Hurdle	Hurdle Height ↑	0.05	0.40
	Hurdle Depth ↑	0.05	0.10

C. Training Architecture

As shown in Fig. 1, two separate MLP encoders map $\mathbf{E}_t$ and $\mathbf{D}_t$ to 32-dimensional features, concatenated into a 64-dimensional environmental embedding and fused with proprioception before separate actor and critic LSTMs. The critic additionally receives the noise-free simulation elevation map; the actor uses only deployable observations. Conditioned on its recurrent hidden state $\mathbf{h}_t^a$, the actor outputs joint targets and the next vertical-map sensing radius, closing the perception-control loop.

IV. Experiments

A. Experimental Configuration

1) Simulation Environment: All policies are trained using NVIDIA Isaac Gym [32] on a modified NVIDIA RTX 4090 GPU with 48 GB of VRAM, using a 12-DoF Unitree G1 humanoid robot. LiDAR point clouds are synthesized using the simulator from [23], which can accurately model sensor noises and occlusion from static terrains and dynamic robot links.

2) Real-world Environment: The trained policy is then deployed on a physical Unitree G1 robot. A Livox Mid-360 LiDAR is mounted on the head to provide raw point cloud data. State estimation and point cloud registration are handled by Fast-LIO2 [33]. The horizontal elevation map $\mathbf{E}_t$ and the vertical distance map $\mathbf{D}_t$ are extracted from the registered point cloud using the GPU-accelerated pipeline from [3] at 10 Hz. The map generation follows the exact same procedure used in simulation. In all real-world deployment experiments, perception and policy inference run on a local computer with an Intel i7-12700H CPU and an NVIDIA RTX 3060 Laptop GPU with 6 GiB VRAM. The local computer generates and sends joint-angle targets to the robot at 50 Hz.

B. Simulation Experiments

1) Evaluation Setup: All methods are evaluated on the highest difficulty tier of *BarrierTrack*. The benchmark includes all seven terrain primitives, grouped into two categories: terrains that require precise foot placement like Stairs, Drop, Jump and Hurdle, and the others that require traversable-space selection like Beam, Pole and Narrow Gate. For each terrain type, every episode contains five obstacles of the same class with progressively increasing difficulty; obstacle parameters are sampled within the range given in Table II according to the curriculum schedule.

2) Metrics: Robot performance is evaluated by the mean and standard deviations over two metrics: (1) Success Rate (R_{succ}), which is the fraction of discrete obstacles that the robot successfully navigates within a single trial, i.e., the number of obstacles crossed divided by 5. and (2) Normalized Traversal Progress (R_{trav}), which is the forward distance traveled along the course normalized by the total track length. Note that R_{trav} is typically no lower than R_{succ}, as the robot may stumble forward for some distance even after failing to clear an obstacle, thereby accumulating partial traversal progress.

3) Performance Comparison: ADAPT is benchmarked against four baselines to evaluate its navigation capability and computational efficiency. Each method is evaluated across all seven terrain types under three speed commands: $v \in \{0.5, 1.0, 1.5\}$ m/s. For each terrain-speed combination, 10 trials are conducted and the average results are reported. The four selected baselines are:

- PIM [4]: A single-stage training method that uses an elevation map from LiDAR point cloud as exteroceptive input. Contrastive learning aligns the generated elevation map with ground truth.
- Parkour [14]: A two-stage method where a teacher policy is first trained with ground-truth elevation maps and then agile locomotion skills are transferred through supervised distillation to a student policy that only uses depth images.
- TSDF: A camera-based approach that performs 3D reconstruction via nvblox [34] and feeds local grid TSDF values as exteroceptive input for policy training.
- Gallant [21]: A voxel-based approach that provides full 3D awareness but at a higher computational cost.

Table III details the performance comparison, while Table IV(a) presents the associated training costs. ADAPT (Ours) achieves the highest success rates across all terrains while requiring less training time per iteration and lower peak GPU memory than the Gallant and TSDF baselines. The performance of the baselines is consistent with the sensing characteristics within each method. PIM relies on a 2.5D elevation map and shows reduced performance on Pole and Narrow Gate terrains where vertical and lateral clearance cues are important. Parkour uses depth images and performs less reliably on terrains requiring broader spatial coverage, such as Jump, Hurdle and Beam. TSDF provides volumetric awareness, but its performance remains limited on fine-grained terrains although incurring the highest training cost. Gallant improves spatial coverage relative to the other baselines, but its higher-dimensional voxel input requires higher computational cost and achieves lower success rates than ADAPT on most terrains. Overall, ADAPT provides a favorable balance between traversal performance and computation efficiency across various tasks.

Although voxels preserve richer 3D geometry, our Gallant implementation trades sensing coverage for fine resolution under a fixed voxel count and computational budget. Its performance gap widens at high speed, where obstacles entering view trigger abrupt deceleration and occasional imbalance, suggesting insufficient look-ahead for smooth motion adjustment. ADAPT instead adjusts its look-ahead distance to the robot's motion state.

All methods use identical hardware and training settings (parallel environments, rollout length, PPO epochs, and batch settings). Table I compares per-frame, pre-encoding actor exteroceptive dimensions: Gallant uses $32\times32\times40 = 40{,}960$ values and ADAPT uses $21 \times 17 + 13 \times 17 = 578$, a 98.6% reduction. Proprioception, observation history, and critic-only inputs are excluded. Table IV(a) reports wall-clock iteration time and peak single-GPU memory monitored with `nvidia-smi`. Relative to Gallant, ADAPT reduces iteration time from 34.6 to 11.3 s (32.7% of the baseline) and peak memory from 36,054 to 15,870 MiB (56.0% lower).

PIM has the lowest training cost but lacks body-clearance information. ADAPT incurs a moderate cost increase over PIM while remaining cheaper than the volumetric baselines.

4) Ablation Study on Adaptive Mechanism: To specifically validate the obstacle avoidance capability of the adaptive mechanism, the ablation study focuses on Beam, Pole and Narrow Gate terrains where perception is critical. For each method, terrain, and speed command, 10 trials are conducted in simulation. Table V reports the success rate averaged over the three terrains.

Besides the proposed method, two additional settings isolating the effect of radius representation and reward formulation are considered: 1) *Fixed Radius*, where the sensing radius r_t is excluded from the observation and action spaces. The policy operates with a constant radius. Separate policies are trained with $R \in \{1, 3, 5\}$ m. 2) *w/o Adaptive Reward*, where r_t remains both an observation and an action, but the `radius_smoothness` and

TABLE III: Benchmarked Comparison in Simulation. R_{succ}: success rate (%, ↑). R_{trav}: normalized traversal progress (%, ↑).

Method	Plane		Stairs		Drop		Jump	
	R_{succ}	R_{trav}	R_{succ}	R_{trav}	R_{succ}	R_{trav}	R_{succ}	R_{trav}
PIM [4]	100 (±0.0)	100 (±0.0)	65.33 (±37.57)	70.00 (±34.61)	99.33 (±3.59)	99.77 (±1.26)	90.67 (±16.92)	91.67 (±14.62)
Parkour [14]	100 (±0.0)	100 (±0.0)	28.00 (±28.10)	37.20 (±24.93)	91.33 (±14.31)	92.40 (±13.52)	44.67 (±20.45)	51.40 (±17.49)
TSDF	100 (±0.0)	100 (±0.0)	40.00 (±20.00)	47.43 (±17.50)	56.67 (±24.81)	63.33 (±22.75)	43.33 (±13.74)	48.63 (±13.65)
Gallant [21]	100 (±0.0)	100 (±0.0)	70.00 (±22.95)	76.30 (±21.75)	80.00 (±31.41)	83.23 (±27.62)	68.00 (±13.27)	74.10 (±11.34)
Ours	**100** (±0.0)	**100** (±0.0)	**94.67** (±13.60)	**95.90** (±10.63)	**100.00** (±0.00)	**100.00** (±0.00)	**98.67** (±4.99)	**98.90** (±4.12)

Method	Hurdle		Beam		Pole		Narrow Gate	
	R_{succ}	R_{trav}	R_{succ}	R_{trav}	R_{succ}	R_{trav}	R_{succ}	R_{trav}
PIM [4]	82.00 (±20.88)	86.37 (±16.44)	66.67 (±9.43)	75.53 (±5.28)	2.67 (±6.80)	6.17 (±7.09)	27.33 (±15.04)	31.17 (±14.17)
Parkour [14]	47.33 (±13.15)	52.67 (±12.81)	14.00 (±11.72)	20.53 (±6.93)	5.33 (±8.84)	15.00 (±7.90)	48.00 (±16.81)	53.03 (±16.05)
TSDF	49.33 (±11.23)	55.47 (±13.98)	33.33 (±11.93)	40.83 (±11.37)	7.33 (±9.64)	14.37 (±9.07)	22.67 (±16.11)	29.80 (±14.31)
Gallant [21]	54.67 (±16.28)	62.47 (±14.21)	32.00 (±25.61)	40.40 (±27.17)	46.00 (±44.17)	49.40 (±41.82)	66.00 (±25.90)	69.57 (±23.35)
Ours	**90.00** (±14.38)	**91.80** (±12.80)	**96.67** (±7.45)	**97.57** (±5.47)	**98.00** (±6.00)	**98.57** (±4.79)	**80.00** (±18.62)	**83.03** (±16.30)

TABLE IV: Training and Runtime Efficiency. Runtime latencies: averages of per-trial means/P95s (ms). ADAPT reports the larger of the two map-stream statistics for each map-latency and perception-to-publish latency entry. Resources: mean whole-host CPU utilization/RAM usage and maximum sampled VRAM usage.

(a) Training efficiency

Method	Train/iter (s)	Peak VRAM (MiB)
PIM [4]	2.6	10485
Parkour [14]	97.8	28791
Gallant [21]	34.6	36054
TSDF	246	38473
ADAPT (Ours)	**11.3**	**15870**

(b) Local-computer runtime latency and resource usage

Method	Map latency Mean/P95 (ms)	Inference latency Mean/P95 (ms)	Perception-to-publish latency Mean/P95 (ms)	Peak VRAM (MiB)	CPU / RAM (% / GiB)
Gallant	64.06 / 87.49	1.91 / 3.20	79.45 / 106.09	5711	38.19 / 10.66
ADAPT	**41.64 / 64.09**	**1.40 / 1.96**	**55.94 / 79.31**	**2373**	**17.76 / 8.04**

`radius_regularization` reward terms are removed. All other reward terms are retained.

Table V summarizes the ablation results. As in the main benchmark, each trial evaluates traversal through a sequence of five obstacles of the same terrain type at a fixed speed command. The success rate R_{succ} is the fraction of the five obstacles successfully crossed in a trial. The table reports the mean and standard deviation over the 30 trial-level success rates pooled across Beam, Pole, and Narrow Gate at each speed.

A short fixed radius ($R = 1$) yields the lowest average success rates, particularly at higher speeds, indicating that the limited look-ahead range is insufficient for these terrains. A long fixed radius ($R = 5$) performs better than $R = 1$ at high speeds but remains inferior to a moderate fixed radius ($R = 3$), suggesting a trade-off between larger look-ahead range and reduced local resolution. However, the performance of a fixed radius is still behind the fully adaptive method. Removing both radius-related rewards in *w/o Adaptive Reward* lowers success at higher speeds, where less time for perception and avoidance may increase sensitivity to unsuitable or fluctuating sensing ranges. Together, the smoothness and regularization terms help stabilize radius selection and its coordination with locomotion. Overall, the fully adaptive method achieves the highest average success rate, indicating efficacy of the proposed closed-loop radius modulation.

TABLE V: Ablation on adaptive sensing radius. R_{succ}: success rate (%). Values are averaged over Beam, Pole, and Narrow Gate terrains.

Type	Config	v=0.5 m/s	v=1.0 m/s	v=1.5 m/s
Fixed Radius	$R = 1$	51.33 (±17.65)	51.33 (±23.49)	24.67 (±22.32)
	$R = 3$	81.33 (±21.25)	72.67 (±15.90)	56.67 (±29.70)
	$R = 5$	64.00 (±18.90)	64.67 (±26.17)	53.33 (±30.26)
w/o Adap. Reward	no r_{adapt}	92.67 (±10.93)	71.33 (±25.66)	42.67 (±29.09)
Adaptive Radius	**Ours**	**94.67** (±10.24)	**96.00** (±8.00)	**84.00** (±19.60)

C. Real-world Experiments

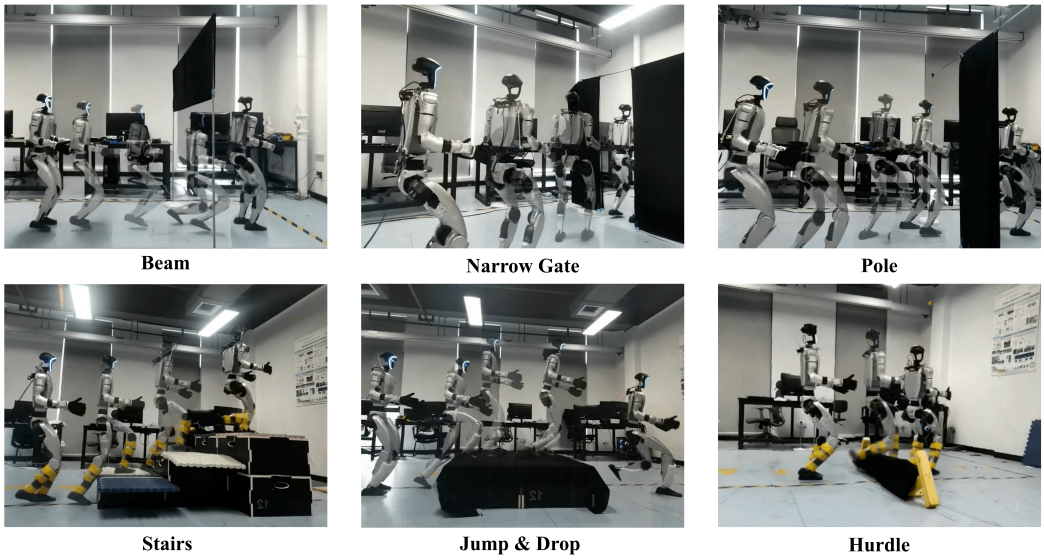

Fig. 4: Zero-shot real-world deployment across diverse terrains. The trained policy enables the physical robot to successfully traverse seven distinct *BarrierTrack* terrains without any real-world fine-tuning.

1) Deployment: ADAPT transfers directly to the Unitree G1 without fine-tuning (Fig. 4). For Fig. 5, final checkpoints from ten independent training seeds are evaluated in matched simulation and real-world scenes with a fixed single-obstacle configuration per terrain. Each policy is tested once per terrain at each speed ($v \in \{0.5, 1.0, 1.5\}$ m/s), totaling 30 trials per terrain and domain.

In terrains that require traversable-space selection, the robot identifies viable regions using the vertical distance map $\mathbf{D}_t$. When encountering aerial obstacles such as beams, the robot lowers its body to crouch underneath the beams and resumes to the normal posture afterward. For narrow pillar-like obstacles, the robot autonomously adjusts its heading to bypass the poles. When facing narrow gates, it precisely aligns its orientation with the traversable path. In terrains requiring precise foot placement, the robot relies on the elevation map $\mathbf{E}_t$ to stably ascend stairs, negotiate drops and jumps, and step over hurdles. The adaptive sensing radius of the vertical distance map further enables the policy to regulate its perception horizon online during high-speed locomotion.

2) Results: Fig. 4 demonstrates zero-shot sim-to-real transfer across all seven terrains, while Fig. 5 presents the corresponding per-terrain success rates in simulation and physical deployment. Despite an inherent sim-to-real domain gap, the real-world success rates closely follow the trend of their counterparts in simulation, maintaining at least 80% success rates across all terrains. The physical robot exhibits a range of whole-body behaviors, including stair ascent and descent, stepping across drops and gaps, crouching under beams of varying height, and navigating through narrow passages and pole fields, suggesting good generality and practical applicability of the learned policy.

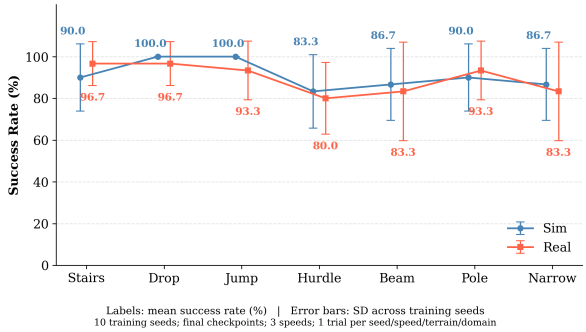

Fig. 5: Single-obstacle traversal success rates in matched simulation and real-world scenes. Points and error bars show the mean and sample standard deviation across ten training seeds, after averaging each policy's outcomes over three speeds.

3) Runtime Efficiency: Table IV(b) compares ADAPT and Gallant on a local computer (i7-12700H, RTX 3060 Laptop GPU with 6 GiB VRAM). The runtime benchmarks use the same local-computer deployment setup as the real-world robot experiments. Each method is tested five times on each of three terrain categories (flat, single obstacle, and complex environment). Map latency spans the input callback to the first observed map-ready event; perception-to-publish latency spans the same input callback to the first command-publication call using that update. Inference latency is measured using current-stream CUDA events. ADAPT reduces mean perception-to-publish latency by 29.6% using the larger of the two stream means and peak GPU memory by 58.4% relative to Gallant, with lower CPU/RAM usage.

V. Conclusions

This paper presented ADAPT, an adaptive dual-projection framework for perceptive humanoid traversal in complex 3D environments. By decomposing local geometry into a horizontal elevation map and a vertical distance map, ADAPT explicitly separates ground-support and body-level clearance constraints into a compact representation for high-frequency control. Furthermore, embedding the sensing radius into the action space enables the robot to dynamically adjust its perceptual horizon based on motion context. Experiments demonstrate that ADAPT significantly improves traversal robustness and computational efficiency over fixed-range and voxel-based alternatives, achieving successful zero-shot sim-to-real transfer on a Unitree G1 robot.

Despite these capabilities, the current framework focuses solely on geometric traversability and adapts only the sensing radius. Future work will investigate richer active perception policies that jointly optimize sensing scale, resolution, and update frequency, while incorporating semantic reasoning to enable long-horizon navigation in complex, human-centered environments.

References

[1] P. Fankhauser, M. Bloesch, C. Gehring, M. Hutter, and R. Siegwart, "Robot-centric elevation mapping with uncertainty estimates," in *International Conference on Climbing and Walking Robots (CLAWAR)*, 2014.

[2] P. Fankhauser, M. Bloesch, and M. Hutter, "Probabilistic terrain mapping for mobile robots with uncertain localization," *IEEE Robotics and Automation Letters (RA-L)*, vol. 3, no. 4, pp. 3019–3026, 2018.

[3] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, "Elevation Mapping for Locomotion and Navigation using GPU," *arXiv e-prints*, p. arXiv:2204.12876, Apr. 2022.

[4] J. Long, J. Ren, M. Shi, Z. Wang, T. Huang, P. Luo, and J. Pang, "Learning humanoid locomotion with perceptive internal model," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 9997–10 003.

[5] H. Wang, Z. Wang, J. Ren, Q. Ben, T. Huang, W. Zhang, and J. Pang, "Beamdojo: Learning agile humanoid locomotion on sparse footholds," *arXiv preprint arXiv:2502.10363*, 2025.

[6] J. Ren, T. Huang, H. Wang, Z. Wang, Q. Ben, J. Long, Y. Yang, J. Pang, and P. Luo, "Vb-com: Learning vision-blind composite humanoid locomotion against deficient perception," *arXiv preprint arXiv:2502.14814*, 2025.

[7] J. He, C. Zhang, F. Jenelten, R. Grandia, M. Bächer, and M. Hutter, "Attention-based map encoding for learning generalized legged locomotion," *Science Robotics*, vol. 10, no. 105, p. eadv3604, 2025.

[8] A. Allshire, H. Choi, J. Zhang, D. McAllister, A. Zhang, C. M. Kim, T. Darrell, P. Abbeel, J. Malik, and A. Kanazawa, "Visual imitation enables contextual humanoid control," *arXiv preprint arXiv:2505.03729*, 2025.

[9] Y. Chen, J. Ma, Z. Luo, Y. Han, Y. Dong, B. Xu, and P. Lu, "Learning Autonomous and Safe Quadruped Traversal of Complex Terrains Using Multi-Layer Elevation Maps," *IEEE Robotics and Automation Letters*, vol. 10, no. 10, pp. 9606–9613, Jan. 2025.

[10] P. Roth, J. Frey, C. Cadena, and M. Hutter, "Learned Perceptive Forward Dynamics Model for Safe and Platform-aware Robotic Navigation," *arXiv e-prints*, p. arXiv:2504.19322, Apr. 2025.

[11] C. Zhang, V. Klemm, F. Yang, and M. Hutter, "AME-2: Agile and Generalized Legged Locomotion via Attention-Based Neural Map Encoding," *arXiv e-prints*, p. arXiv:2601.08485, Jan. 2026.

[12] S. Li, S. Luo, J. Wu, and Q. Zhu, "MOVE: Multi-skill Omnidirectional Legged Locomotion with Limited View in 3D Environments," *arXiv e-prints*, p. arXiv:2412.03353, Dec. 2024.

[13] Z. Zhuang, Z. Fu, J. Wang, C. G. Atkeson, S. Schwertfeger, C. Finn, and H. Zhao, "Robot parkour learning," *arXiv preprint arXiv:2309.05665*, 2024.

[14] Z. Zhuang, S. Yao, and H. Zhao, "Humanoid parkour learning," *arXiv preprint arXiv:2406.10759*, 2024.

[15] R. Yu, Q. Wang, H. Li, Z. Jun, Z. Wang, J. Wu, and Q. Zhu, "START: Traversing Sparse Footholds with Terrain Reconstruction," *arXiv e-prints*, p. arXiv:2512.13153, Dec. 2025.

[16] G. A. Castillo, H. Lodha, and A. Hereid, "Learning Terrain Aware Bipedal Locomotion via Reduced Dimensional Perceptual Representations," *arXiv e-prints*, p. arXiv:2512.12993, Dec. 2025.

[17] J. Solano and D. Quiroz, "Stand, Walk, Navigate: Recovery-Aware Visual Navigation on a Low-Cost Wheeled Quadruped," *arXiv e-prints*, p. arXiv:2510.23902, Oct. 2025.

[18] J. Sun, G. Han, P. Sun, W. Zhao, J. Cao, J. Wang, Y. Guo, and Q. Zhang, "DPL: Depth-only Perceptive Humanoid Locomotion via Realistic Depth Synthesis and Cross-Attention Terrain Reconstruction," *arXiv e-prints*, p. arXiv:2510.07152, Oct. 2025.

[19] Y. Wang and G. Tao, "LocoMamba: Vision-Driven Locomotion via End-to-End Deep Reinforcement Learning with Mamba," *arXiv e-prints*, p. arXiv:2508.11849, Aug. 2025.

[20] S. Zhu, Z. Zhuang, M. Zhao, K.-Y. Lee, and H. Zhao, "Hiking in the Wild: A Scalable Perceptive Parkour Framework for Humanoids," *arXiv e-prints*, p. arXiv:2601.07718, Jan. 2026.

[21] Q. Ben, B. Xu, K. Li, F. Jia, W. Zhang, J. Wang, J. Wang, D. Lin, and J. Pang, "Gallant: Voxel Grid-based Humanoid Locomotion and Local-navigation across 3D Constrained Terrains," *arXiv e-prints*, p. arXiv:2511.14625, Nov. 2025.

[22] S. Niijima, R. Tsuzaki, N. Takasugi, and M. Kinoshita, "Real-time Multi-Plane Segmentation Based on GPU Accelerated High-Resolution 3D Voxel Mapping for Legged Robot Locomotion," *arXiv e-prints*, p. arXiv:2510.01592, Oct. 2025.

[23] Z. Wang, T. Ma, Y. Jia, X. Yang, J. Zhou, W. Ouyang, Q. Zhang, and J. Liang, "Omni-Perception: Omnidirectional Collision Avoidance for Legged Locomotion in Dynamic Environments," *arXiv e-prints*, p. arXiv:2505.19214, May 2025.

[24] H. Tang, Z. Liu, S. Zhao, Y. Lin, J. Lin, H. Wang, and S. Han, "Searching Efficient 3D Architectures with Sparse Point-Voxel Convolution," *arXiv e-prints*, p. arXiv:2007.16100, July 2020.

[25] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast Encoders for Object Detection from Point Clouds," *arXiv e-prints*, p. arXiv:1812.05784, Dec. 2018.

[26] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *arXiv e-prints*, p. arXiv:2201.08117, Jan. 2022.

[27] H. Xue, S. Liang, Z. Zhang, Z. Zeng, Y. Liu, Y. Lian, J. Wang, Q. Liu, X. Shi, and L. Yi, "Collision-Free Humanoid Traversal in Cluttered Indoor Scenes," *arXiv e-prints*, p. arXiv:2601.16035, Jan. 2026.

[28] Y. Wang, Y. Yue, Y. Yue, H. Wang, H. Jiang, Y. Han, Z. Ni, Y. Pu, M. Shi, R. Lu, Q. Yang, A. Zhao, Z. Xia, S. Song, and G. Huang, "Emulating Human-like Adaptive Vision for Efficient and Flexible Machine Visual Perception," *arXiv e-prints*, p. arXiv:2509.15333, Sept. 2025.

[29] Y. Zhang, J. Ma, L. Yan, Z. Cao, Y. Zhang, H. Li, and Y. Gao, "FocusNav: Spatial Selective Attention with Waypoint Guidance for Humanoid Local Navigation," *arXiv e-prints*, p. arXiv:2601.12790, Jan. 2026.

[30] M. Patel, J. Frey, D. Atha, P. Spieler, M. Hutter, and S. Khattak, "RoadRunner M&M - Learning Multi-Range Multi-Resolution Traversability Maps for Autonomous Off-Road Navigation," *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 425–11 432, Jan. 2024.

[31] Z. Liu, Y. Gu, Y. Wang, X. Xue, and Y. Fu, "ActiveVLA: Injecting Active Perception into Vision-Language-Action Models for Precise 3D Robotic Manipulation," *arXiv e-prints*, p. arXiv:2601.08325, Jan. 2026.

[32] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning," *arXiv e-prints*, p. arXiv:2108.10470, Aug. 2021.

[33] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast Direct LiDAR-Inertial Odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, Jan. 2022.

[34] A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, and R. Siegwart, "nvblox: GPU-Accelerated Incremental Signed Distance Field Mapping," *arXiv e-prints*, p. arXiv:2311.00626, Nov. 2023.